AI 原生軟體開發流程：實際怎麼運作

一頁看懂：寫程式變快之後，整個開發流程也要跟著改，關鍵決定仍由人來做

1 現況

AI 寫程式的速度比一年前快很多，但需求、審查、上線還是人工速度，整體交付時間沒有等比例縮短。

2 做法

把「規劃、設計、開發、測試、上線、維運」串成一個循環。每個階段結束都留下一份檔案，下一個階段從這份檔案接手。

3 把關

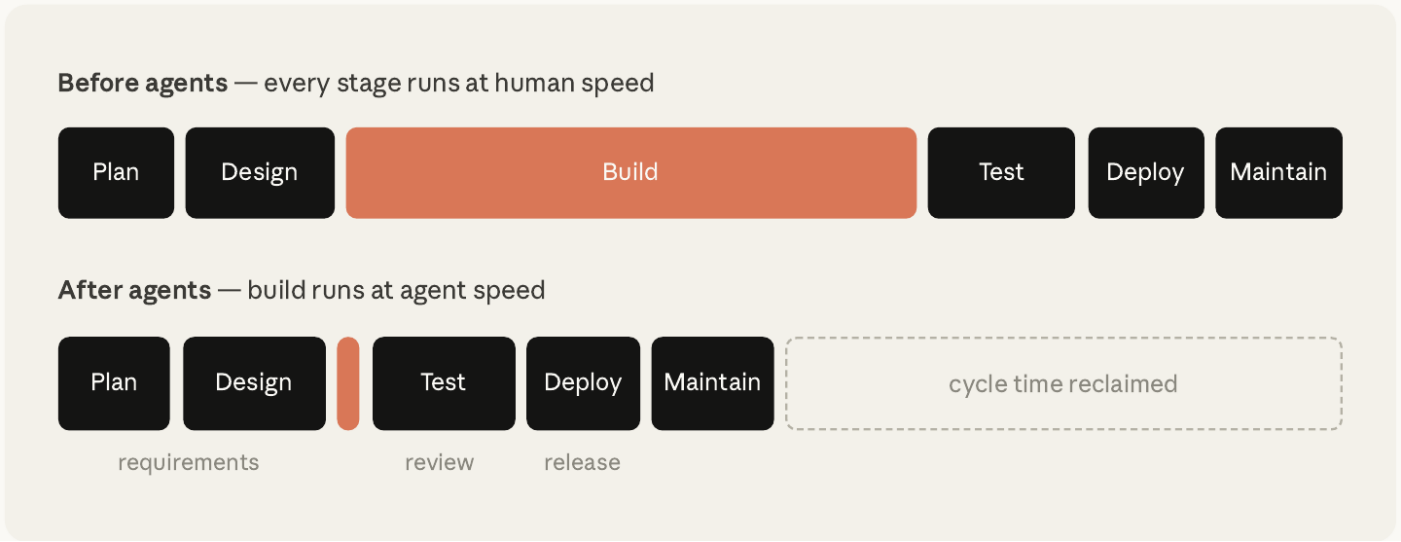
AI 負責起草、檢查與執行。接受需求、核准計畫、核准上線這類需要判斷的決定，一律由指定的人做，而且都有紀錄。

對客戶的三個好處	
更快	手冊預期，從想法到可執行的需求，可由數週縮短到數小時。
查得到	誰提出、 AI 產出什麼、誰核准，全部寫在版本紀錄裡，就是現成的稽核軌跡。
守得住	公司規範寫成 AI 會照做的規則；正式上線前一定要有人授權。

給客戶的一句話：「 AI 幫你把每一步做得更快，每一步也都留下紀錄；要不要上線，永遠由你的人決定。」

來源： Anthropic 《 The AI-Native SDLC Playbook 》，第 3–7 頁。

為什麼要改： AI 把「寫程式」縮短到幾小時，等待時間移到了前後的人工步驟



圖： Anthropic 《 The AI-Native SDLC Playbook 》 圖 1 「 The bottleneck moved 」

這張圖怎麼看

上排「 Before agents 」：
過去每個階段都靠人工，其中開發（橘色 Build ）最花時間。
下排「 After agents 」：
AI 讓開發縮成一小段，但規劃、設計、測試、上線、維運的長度都沒變。右邊虛線框是「可以拿回來的時間」，要靠流程一起改才拿得到。

圖中英文： Plan 規劃 | Design 設計 | Build 開發 | Test 測試 | Deploy 上線 | Maintain 維運

1 瓶頸移位
需求、審查、上線還是人工速度，整體時間被這些步驟卡住。

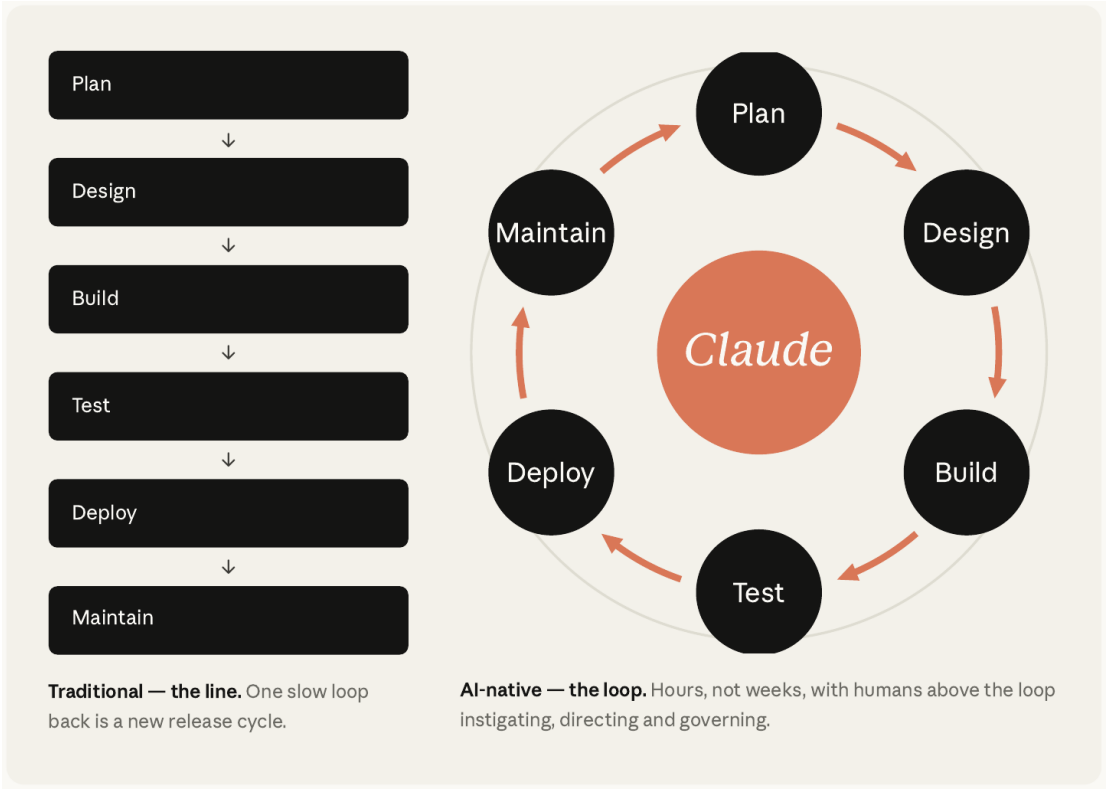
2 控制跟不上
逐行人工審查，在人寫程式的年代合理； AI 寫出大部分程式後就追不上。

3 治理成本上升
例外事項還是要等每週或每月的會議， AI 做得越快，排隊越長。

資安審查的例子：資安人力是依人寫程式的產量配置。程式量倍增後，不是審查排隊變長，就是沒審完就上線。受監理的機構兩者都不能接受，所以檢查也必須跟上 AI 的速度。

來源： Anthropic 《 The AI-Native SDLC Playbook 》，第 3–4 頁。

從「一條直線」變成「一個循環」：每一步自動交棒，人在循環上方發起、指揮與把關



圖： Anthropic 《 The AI-Native SDLC Playbook 》 圖 2 「 The line and the loop 」

這張圖怎麼看

左邊「 Traditional — the line 」傳統的直線：
六個階段由不同角色依序負責，靠文件、工單、簽核交接。只要退回一次，就等於重新開始一個版本週期。

右邊「 AI-native — the loop 」 AI 原生的循環：
AI （圖中的 Claude ）在中間參與每一個階段。上一步核准的檔案直接觸發下一步，手冊的說法是「以小時計，不是以週計」。

人在循環上方：
人負責發起、指揮與治理，專心看 AI 標出來的重點，不必每一步從頭做起。

白話：以前是「做完一段、交給下一組人、等簽核」；現在每一段做完就把檔案交出去，下一段立刻接手，人只在關卡上做決定。

來源： Anthropic 《 The AI-Native SDLC Playbook 》，第 5 、 8–9 頁。

六個階段白話對照：做法都變了，但每一步都留下一份下一步能接手的紀錄

階段	以前怎麼做	現在怎麼做（AI 原生）	留下的紀錄	誰拍板
規劃	需求要經過多次工作坊與簽核，再由專人寫成文件	提出想法的人直接和 AI 討論，整理成一份需求說明檔	intent.md 需求說明	產品負責人
設計	分析師寫需求，設計師再轉成設計，兩組人分開做	AI 依需求說明一次產出需求與設計規格，套用資安、法遵、品牌規範，並標出疑慮	spec.md 規格書	產品負責人（高風險找技術主管）
開發	工程師讀完設計就寫程式，做法只在自己腦中	AI 先寫實作計畫，工程師修正、核准後才動手；團隊慣例寫成 AI 每次都會讀的說明檔	plan.md 實作計畫 程式與測試	工程師（高風險找架構師）
測試	階段結束才由 QA 把關，問題很晚才發現	AI 每改一次就自己跑測試，通過才交給人；AI 的設定改了，也要重跑評測	測試結果 建置紀錄	程式負責人
上線	每一行程式都由人審，品質看審查者忙不忙	AI 先做多輪審查並排出嚴重度，人專注判斷意圖與風險；正式上線一定要人授權	PR 審查紀錄 核准紀錄	程式負責人、發布經理
維運	人盯著系統，事件常被擱置	監控程式發現異常才請 AI 診斷，結果寫成新的需求說明，重新進入循環	事件紀錄 新的 intent.md	服務負責人

來源：Anthropic《The AI-Native SDLC Playbook》，第 6 頁「The shifts」及各階段說明。

實際走一遍（手冊範例）：「理賠進度自助查詢」從客服痛點到上線的七個步驟

1 理賠作業人員	2 產品負責人	3 AI ＋產品負責人	4 工程師＋ AI	5 AI	6 AI 審查＋人核准	7 監控＋ AI
向 AI 描述問題：客戶常打電話問理賠進度，約三分之一通話時間花在查進度	檢查並修正 AI 整理的需求說明，決定是否接受	AI 產出規格並套用資安規範（入口網站不新增個資）；疑慮交給規範負責人處理	AI 寫實作計畫：改哪些檔案、順序、風險（核心系統每秒 50 次請求上限，要加快取）	照計畫寫程式並自己跑測試，四種理賠狀態都要通過，畫面截圖比對設計稿	AI 依審查規則查錯誤、資安、合規；程式負責人核准合併；發布經理授權上線	錯誤率超出正常範圍時，AI 先診斷，必要時執行事先演練的回復，寫成新的需求說明
intent.md	**接受紀錄**	**spec.md**	**plan.md**	**程式＋測試結果**	**審查與核准紀錄**	**事件紀錄**

BA 的觀察：整個過程中，人做了三件事：說清楚問題（第 1 步）、決定要不要做（第 2 步）、決定能不能上線（第 6 步）。其餘起草、計畫、寫程式、測試、初審都由 AI 先做，再交給人看。

深橘為人做決定的步驟。數字（三分之一通話時間、每秒 50 次）為手冊示例，非實際客戶資料。來源： Anthropic 《The AI-Native SDLC Playbook》，第 12–20 、 48–52 頁。

手冊範例原文：需求說明與實作計畫長什麼樣子（附中文翻譯）

需求說明（intent.md）：理賠部門同仁和 AI 討論後產出

```
# Intent: claims status self-service
Author: J. Ortiz (claims operations). Status: draft.

## Problem
Customers phone the contact center to ask where their claim is.
Handlers spend roughly a third of call time on status-only queries.

## Proposed outcome
Customers see claim status, next step and expected date in the portal.

## Affected users and systems
Claims handlers, portal team, claims-core API.

## Constraints
No new PII in the portal session. Existing authentication only.

## Open questions
Do third-party loss adjusters need access too?
```

中文翻譯

標題：理賠進度自助查詢｜提出者：理賠作業 J. Ortiz ｜狀態：草稿
問題：客戶打電話問理賠進度，處理人員約三分之一通話時間只在回答進度。
期望結果：客戶在網站上看到理賠狀態、下一步與預計日期。
影響對象：理賠人員、網站團隊、理賠核心系統 API。
限制：網站不新增個資，只用現有登入方式。
待釐清：外部理賠公估人是否也需要使用？

實作計畫（plan.md）：工程師核准後才開始寫程式

```
# Plan: claims status self-service (from intent.md 2026-06-02)

## Files that change
portal/src/claims/StatusPanel.tsx (new), claims-api/routes/status.py,
claims-api/tests/test_status.py

## Order of work
1. Add the status endpoint behind existing auth.
2. Panel against the endpoint.
3. Wire into the portal nav.

## Risks
The claims-core API rate-limits at 50 rps; the panel must cache.

## Proof
test_status.py covers the four claim states; screenshot matches the
approved mock.
```

中文翻譯

標題：理賠進度自助查詢（依 2026-06-02 的需求說明）
要改的檔案：網站理賠狀態面板（新增）、理賠 API 路由與測試。
工作順序：① 在現有驗證下新增狀態 API ② 面板串接 API ③ 放進網站選單。
風險：理賠核心 API 每秒最多 50 次請求，面板必須做快取。
完成證明：測試涵蓋四種理賠狀態；畫面截圖與核准的設計稿一致。

圖： Anthropic 《The AI-Native SDLC Playbook》 第 13 、 20 頁。兩份檔案都是一般文字，產品負責人和 AI 都能直接閱讀與修改；內容為手冊示例，非客戶資料。

每一步都留下紀錄：串起來就是一條完整的稽核軌跡，回答「誰要了什麼、誰核准」

intent.md 需求說明	spec.md 規格書	plan.md 實作計畫	程式 + 測試 改動本身	PR 審查 審查紀錄	事件紀錄 維運紀錄
誰提出、什麼時候、要解決什麼	套用了哪些規範、哪些疑慮怎麼處理	誰核准了做法與風險	改了什麼、測試是否通過	AI 找到什麼、人怎麼決定	發生什麼、怎麼處理、學到什麼

已經有 Jira 、 ServiceNow 或需求管理系統，怎麼辦？

以程式庫為準	需求說明、規格等檔案是正式紀錄，舊系統放連結。 一個工具、一個時間戳記，最乾淨。
以現有系統為準	Jira 等系統是正式紀錄， AI 開工時讀取、做完寫回； 檔案只是工作副本。
至少互相連結（起步建議）	檔案記下工單編號，工單記下檔案版本號。接受兩個來源並存，先讓兩邊能對上。

稽核時可以直接回答的三個問題

1. **誰要求了什麼？**
   需求說明檔的作者、時間與修改歷程
2. **AI 產出了什麼？**
   規格、計畫、程式與測試結果，以及當時套用的規範版本
3. **誰核准？**
   每個關卡的核准人與時間，記在合併與審查紀錄中

不需要汰換現有系統，重點是每一種紀錄都說清楚「以哪個系統為準」。來源： Anthropic 《 The AI-Native SDLC Playbook 》，第 6–7 、 22 頁。

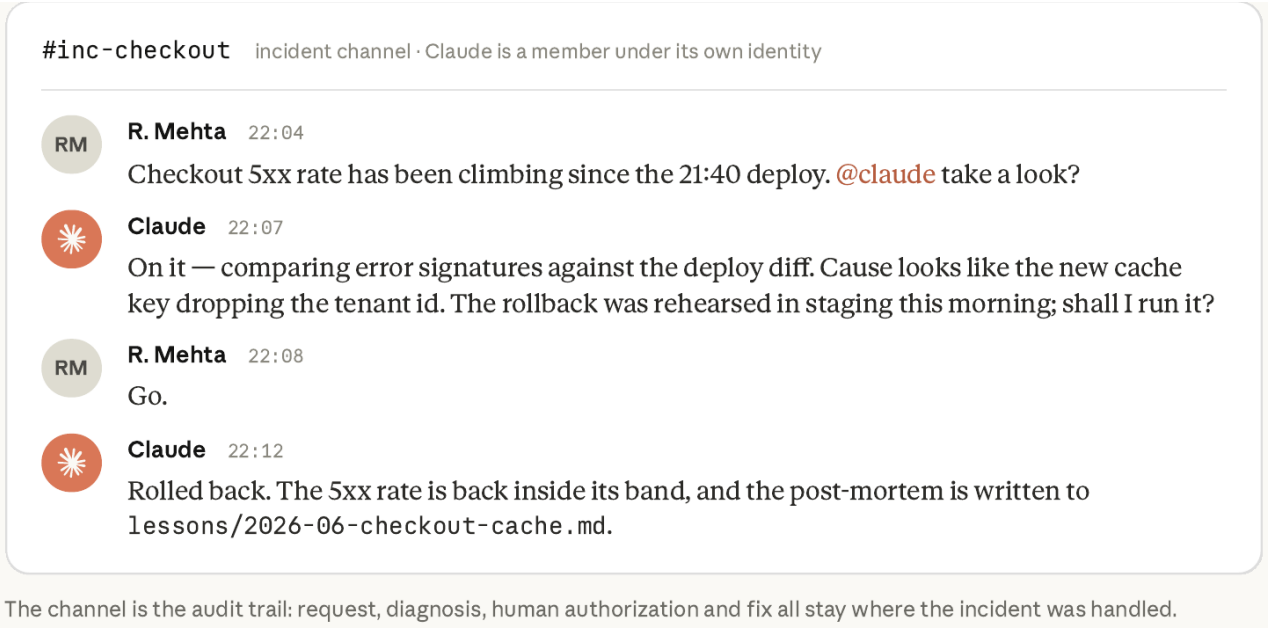
人在哪裡把關：五個關鍵決定由指定的人做， AI 寫的程式不能自己核准

關卡	誰決定	決定什麼	怎麼確保一定經過這一關
① 接受需求	產品負責人	這個想法值不值得做	接受或關閉需求說明，留下紀錄
② 確認規格	產品負責人（高風險找技術主管）	規格有沒有解決原本的問題，標出的疑慮是否已找規範負責人處理	規格與需求說明一起存檔
③ 核准計畫	工程師（高風險找技術主管或架構師）	要怎麼改、最大的風險在哪	計畫模式下，核准前 AI 不能修改任何檔案
④ 核准合併	程式負責人	改動是否符合原意、風險能否接受	分支保護：沒有程式負責人核准就不能合併
⑤ 授權上線	發布經理	能不能進入正式環境	上線閘門：沒有授權，上線指令會被擋下

原則： AI 可以一路做到正式環境的門口，但不能自己開門。寫程式的 AI 沒有核准權限，職責分離和現在一樣。

來源： Anthropic 《 The AI-Native SDLC Playbook 》，第 13 、 17 、 20 、 39 、 46 頁。

上線後出事怎麼辦： AI 在事件頻道當第一線回應，但「要不要回復」由人說了算



圖： Anthropic 《 The AI-Native SDLC Playbook 》 圖 4 「 A first responder in the incident channel 」

對話中文翻譯

22:04　R. Mehta （工程師）
結帳頁的 5xx 錯誤率從 21:40 上版後一直上升， @claude 幫忙看一下？

22:07　Claude
處理中，正在比對錯誤特徵與這次上版的差異。原因看起來是新的快取鍵漏掉了租戶 ID 。今天早上已在測試環境演練過回復，要執行嗎？

22:08　R. Mehta （工程師）
執行。

22:12　Claude
已回復。 5xx 錯誤率回到正常範圍，事後檢討已寫入 lessons/2026-06-checkout-cache.md 。

重點： AI 先診斷原因並提出建議，但只有工程師回覆「執行」之後才回復上一版。請求、診斷、人的授權、處理結果都留在同一個頻道，頻道本身就是稽核軌跡；事後檢討寫成檔案，下次遇到類似事件可以直接參考。

Claude Tag 在手冊撰寫時為 Slack 上的公開測試版，對客戶說明前先確認現況。來源： Anthropic 《 The AI-Native SDLC Playbook 》，第 52 頁。

控制機制白話版：規範手冊讓 AI 照著做，自動閘門讓違規做不到

技能（Skill） = AI 會讀的公司規範手冊

資安、API、品牌規範寫成檔案，AI 寫程式時自動套用。規範改了，大家下一次使用就是新版。

建議性：讓違規變少

鉤子（Hook） = AI 動作前的自動閘門

AI 每個動作前先檢查，可以放行、要求核准或擋下。例如沒有變更單不能改基礎設施，修錯時不能改測試。

強制性：讓違規幾乎做不到

集中管理設定 = IT 統一鎖定的規則

禁止讀取密碼與金鑰檔、限制可連的網站、只用公司核准的外掛。工程師無法自行關閉。

由 IT 管理，個人改不動

分環境授權 = 環境越重要，AI 權限越小

開發環境 AI 可以自由部署；正式環境由 AI 準備、發布經理授權。退回上一版是最常演練的流程。

正式環境一定要人授權

手冊原話：技能讓違規變少，鉤子讓違規幾乎不可能。來源：Anthropic《The AI-Native SDLC Playbook》，第 25–27、40–46 頁。

客戶最常問的五個問題，以及可以怎麼回答

客戶問	可以這樣回答
會不會失控？	AI 只能走核准過的路徑。正式上線一定要指定的人授權，每一次放行或阻擋都有時間戳記。
出事誰負責？	每個需要判斷的決定都由人做，並記在版本紀錄中。寫程式的 AI 不能核准自己的程式，職責分離不變。
會和現有 Jira、變更管理衝突嗎？	不需要汰換。每種紀錄指定一個「以誰為準」的系統，其他系統放連結；起步可先做互相連結。
機密資料安全嗎？	IT 可統一禁止 AI 讀取金鑰與機密檔、限制對外連線、在隔離環境執行。也可透過 AWS Bedrock、Google Vertex 或 Microsoft Foundry 使用，留在公司既有的雲端合約內。
怎麼知道有沒有效？	用版本紀錄與 CI 系統本來就有的資料量測，例如需求定稿時間、第一次就通過測試的比例、審查等待時間、上線失敗率。

來源：Anthropic《The AI-Native SDLC Playbook》，第 22、37、41–46 頁。

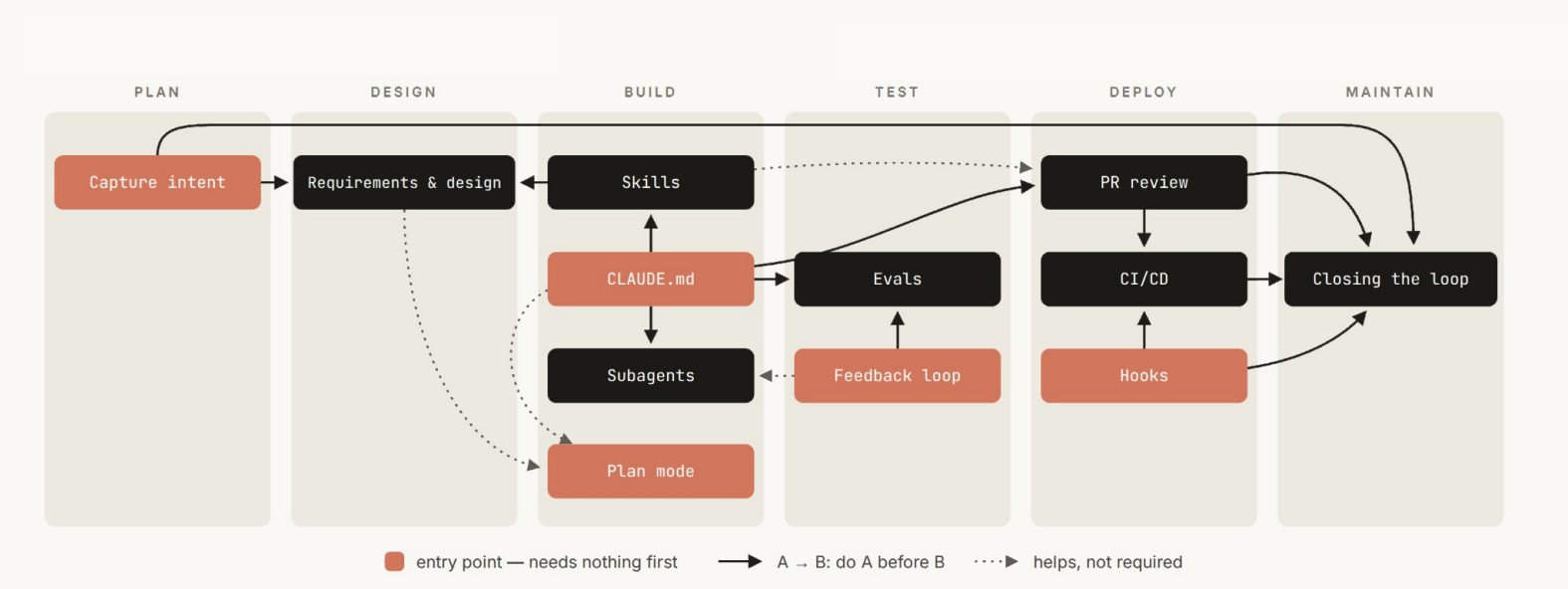
怎麼衡量成效：數據大多已經在版本紀錄和 CI 系統裡，不需要另外填表

階段	先看（領先指標）	後看（落後指標）
規劃	從第一次討論到需求說明定稿的時間（手冊預期由數週降到數小時）	被接受、進入設計的需求比例
設計	需求說明到規格定稿的時間	開發開始後還要改規格的次數
開發	第一次實作就能合併的比例	每個改動的返工次數
測試	AI 改動第一次就通過 CI 的比例	每個 PR 的審查時間、上線失敗率
上線	第一次審查的等待時間（目標降到分鐘級）	合併前抓到的缺陷，對比上線後才發現的
維運	異常發生到寫成需求說明的時間	同類事件重複發生的次數

建議：試點前先記下現況基準，試點後用同一套指標比較，對外只引用自己量到的數字。

整體交付績效可再搭配業界常用的 DORA 指標（部署頻率、交付時間、變更失敗率、復原時間）。來源： Anthropic 《The AI-Native SDLC Playbook》，各階段「How to measure it」。

導入順序圖：橘色項目不需要任何前置條件，可以先做；箭頭代表誰要先於誰



圖： Anthropic 《The AI-Native SDLC Playbook》 圖 3 「The play dependency graph」

怎麼讀：

圖由左到右分成規劃、設計、開發、測試、上線、維運六欄。橘色方塊是入口，不需要先做任何事就能開始；實線箭頭表示「A 要先於 B」；虛線表示「有幫助但不是必要」。

例如： PR 審查要等團隊說明檔與技能到位，閉環維運則要等審查、鉤子與 CI/CD 都準備好，所以放在最後。

圖中方塊中文對照

Capture intent	記錄需求（需求說明）
Requirements & design	需求與設計規格
Skills	技能（規範手冊）
CLAUDE.md	團隊說明檔
Subagents	子代理
Plan mode	計畫模式
Evals	持續評測
Feedback loop	AI 自我驗證
PR review	PR 審查
CI/CD	自動化流水線
Hooks	鉤子（自動閘門）
Closing the loop	閉環維運

來源： Anthropic 《The AI-Native SDLC Playbook》，第 8–9 頁。

怎麼開始：先做不需要前置條件的項目，再一步一步把循環接起來

第一步｜先做
不需要任何前置條件

- 需求說明範本（intent.md）
- 團隊說明檔（CLAUDE.md）
- 把一條常被漏掉的規範寫成技能
- 讓 AI 自己跑測試再交件
- 用鉤子守住現有的核准關卡

第二步｜串起來
以第一步為基礎

- 設計規格由 AI 起草、人審核
- 計畫模式成為開發的起點
- AI 參與 PR 審查
- AI 設定變更要跑持續評測

第三步｜自動化
關卡都到位之後

- AI 進入 CI/CD（先唯讀，再寫入）
- 監控觸發 AI 診斷，結果回到規劃
- 事件頻道裡由 AI 擔任第一線回應

第一個試點怎麼挑：
依手冊建議，挑「規格清楚、影響範圍小、已經有測試覆蓋」的日常工作。先用人工逐步帶著 AI 做，關卡穩定後，再讓核准過的檔案自動觸發下一步。
提醒：自動化要在核准關卡都到位之後才做，否則只是讓沒把關的東西更快上線。

各項目的先後順序見上一頁導入順序圖。來源： Anthropic 《The AI-Native SDLC Playbook》，第 8–9 、 21 、 44 頁。

BA 翻譯小抄（１／２）：工程團隊常用詞，對管理層和客戶可以這樣說

工程用語	白話說法
SDLC	軟體從想法到上線、再到維護的整套流程
intent.md （需求說明）	提出者用自己的話寫下：要什麼、為什麼、有什麼限制
spec.md （規格書）	AI 依需求說明和公司規範起草，人負責審核
plan.md （實作計畫）	要改哪些地方、順序、風險、怎麼證明完成
CLAUDE.md （團隊說明檔）	新進同仁第一天要知道的事， AI 每次開工都先讀
Skill （技能）	AI 會自動套用的公司規範手冊
Hook （鉤子）	AI 動作前的自動檢查站：放行、詢問或擋下
Plan mode （計畫模式）	AI 只能讀不能改，等人核准計畫才動手
Auto mode （自動模式）	計畫核准後， AI 不必每改一處都問人
PR （ Pull request ）	改動送審單：附上改了什麼、測試結果，等人核准
Code review （程式審查）	AI 先查錯誤與資安，人判斷意圖與風險

用語依手冊原文整理； Claude Tag 、 Claude Design 等產品名稱為手冊撰寫時的預覽或測試版功能，對客戶說明前先確認現況。

BA 翻譯小抄（２／２）：工程團隊常用詞，對管理層和客戶可以這樣說

工程用語	白話說法
CI/CD	自動建置、測試、部署的流水線
Eval （評測）	用一組真實任務考 AI ，確認換模型或改設定後品質沒有退步
Worktree ／平行工作階段	一位工程師同時開幾個 AI 分頭做不同工作，彼此不干擾
Subagent （子代理）	專做一件事的小幫手，例如驗證程式能不能正常執行
MCP	讓 AI 安全連接其他系統（ Jira 、部署工具）的標準介面
Sandbox （沙箱）	隔離環境： AI 在裡面執行，碰不到不該碰的檔案和網路
Managed settings （集中管理設定）	IT 統一下發、工程師不能改的設定
Control band （ σ ）	指標的正常波動範圍，超出越多， AI 能做的處理越多
Rollback （回復）	一個指令退回上一版，平時就要演練
DORA 指標	業界常用的交付績效指標：部署頻率、交付時間、變更失敗率、復原時間
OpenTelemetry	把 AI 每個動作記錄下來、送到監控系統的標準格式

用語依手冊原文整理； Claude Tag 、 Claude Design 等產品名稱為手冊撰寫時的預覽或測試版功能，對客戶說明前先確認現況。

對客戶怎麼講： 30 秒版本、可以強調的重點，以及要避免的說法

30 秒版本

「現在 AI 寫程式很快，但如果需求、審查、上線還照舊，整體不會快多少。我們協助你把開發流程重新設計： AI 參與每一步，每一步都留下紀錄，關鍵決定仍由你的人做。先從一個小系統試，用你們既有的版本紀錄和 CI 數據量測成效。」

可以強調

- 每一步都有紀錄，稽核時可以直接調出來
- 寫程式的 AI 不能核准自己的程式，職責分離不變
- 可沿用現有的 Jira 、 ServiceNow 與變更管理
- 從不需要前置條件的項目開始，風險小
- 正式上線一定要指定的人授權

避免的說法 → 改成

「AI 會取代工程師」	**工程師的工作轉為規劃、指揮與審查**
「以後不用人審了」	**人審集中在高風險與受監理的程式**
「保證從數週變數小時」	**手冊預期可大幅縮短，實際以試點量測**
「AI 會自己上線」	**AI 只做到上線前，授權一定由人來做**

推案前：客戶若同時是所內審計客戶，先與 Risk & Quality 確認獨立性；客戶資料不放進任何外部 AI 工具試用。

來源與延伸閱讀

主要來源： Louis Claxton （Anthropic），《The AI-Native SDLC Playbook》，2026。本簡報的流程、範例與控制機制皆整理自該手冊，並改寫為白話中文。

手冊列出的官方文件（依導入順序，可點擊）

1. 組織導入總覽（先看這份）
2. 設定與優先順序（含集中管理設定）
3. 權限設定
4. 沙箱：檔案與網路隔離
5. 鉤子（Hooks）指南
6. 技能（Skills）
7. 外掛與私有市集
8. 集中管理 MCP
9. 企業部署：Bedrock、Vertex、Foundry
10. 監控（OpenTelemetry）
11. 安全模型
12. 合規 API

連結取自手冊第 54 頁；本工作環境無法開啟網頁，使用前請點開確認。